

COMP3069 Computer Graphics

Lab 8

Originally prepared by Dr Kristian Spoerer of UNUK

Modified by Prof Sean He of UNNC

Learning Outcomes

After Lab 8 you are expected to be able to

1. Specify UV texture coordinates and setup a VAO correctly, to map a texture in various ways to a quad.
2. Write shaders to perform simple texture mapping.
3. Use a provided API function to load bitmap files.
4. Use OpenGL code to create and bind a texture object, and set pixel data, in order to render texture mapped geometry.
5. Demonstrate that you can map a texture to geometry and render it.
6. Recognise texture pixelation caused by magnification.
7. Use OpenGL Bi-linear filtering to smooth pixelation caused by magnification.
8. Compare the difference between Bi-linear filtering enabled and disabled.
9. Recognise texture aliasing caused by minification.
10. Use OpenGL code to manually specify mipmaps to a texture object.
11. Use OpenGL Tri-linear filtering to smooth texture aliasing, using mipmaps.
12. Compare the difference between Tri-linear filtering enabled and disabled.
13. Use an OpenGL function to generate mipmaps.
14. Analyse the difference between manual or automatic mipmaps when saving time or space.

Setup

In the COMP3069 directory, create a new project called

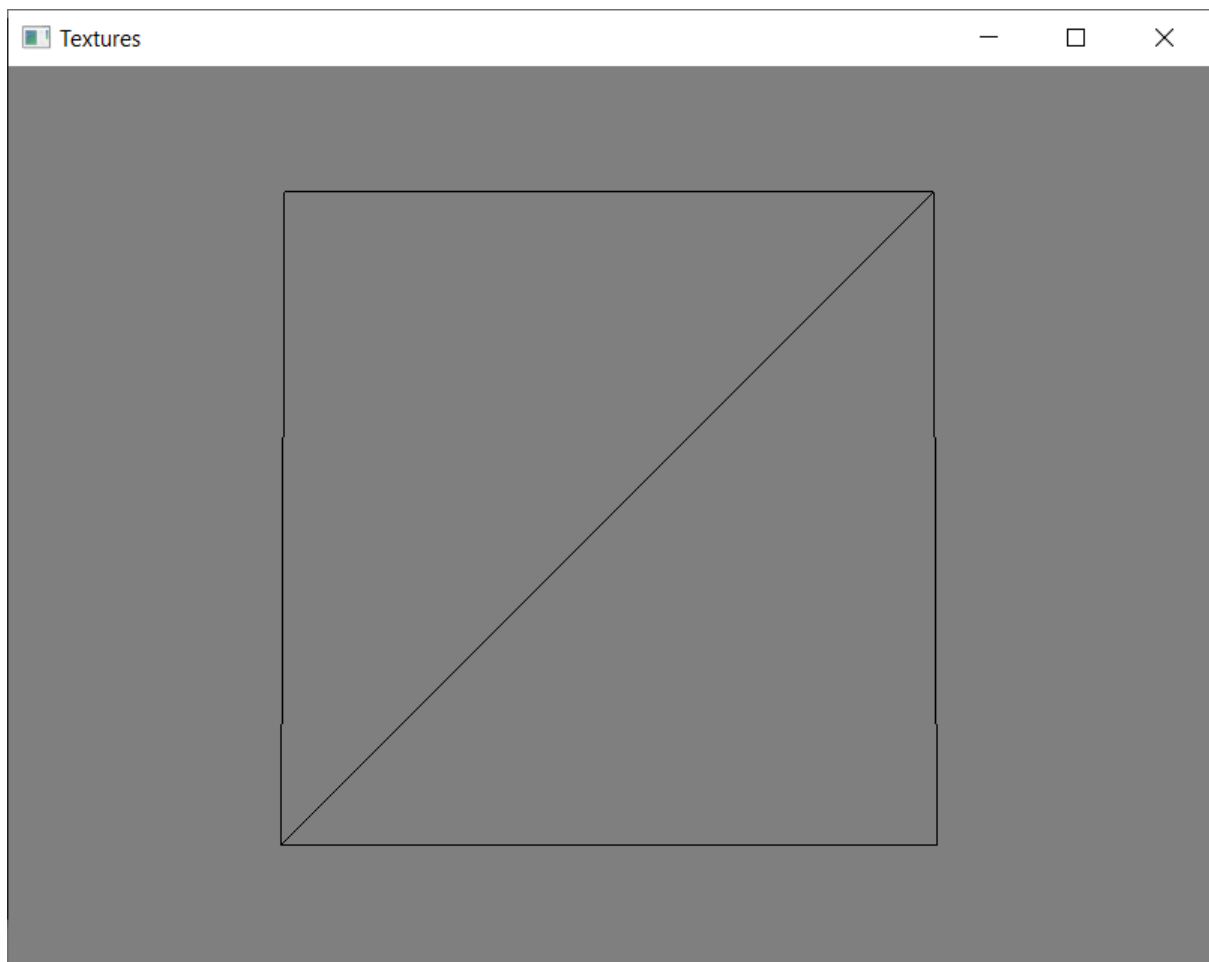
Lab8

Download Lab8.zip from Moodle, unzip the file, and copy the code in the file Lab8.cpp to the Lab8.cpp in your new project.

Copy `bitmap.h`, `camera.h`, `ModelViewerCamera.h`, `shader.h`, `texture.h`, `util.h`, and `window.h` to your project, and add them to the Header Files in the Solution Explorer.

Copy the vertex shader `texture.vert` and fragment shader `texture.frag` to your project, and add them to the Source Files in the Solution Explorer.

If you run the program, then the following window should appear



There are two triangles rendered as wireframe, making a quad. In this lab, we are going to map textures to this quad.

Specify the Texture coordinates

Copy

jubilee.bmp

to your project directory where the shader files are.

The first thing you need to do is to specify the texture UV coordinates in the vertex array in Lab8.cpp

```
float vertices[] =  
{  
    //t1  
    //pos  
    -0.9f, 0.9f, 0.f,  
    0.9f, 0.9f, 0.f,  
    0.9f, -0.9f, 0.f,  
    //t2  
    //pos  
    -0.9f, 0.9f, 0.f,  
    0.9f, -0.9f, 0.f,  
    -0.9f, -0.9f, 0.f,  
};
```

Have a look at Lab 2, and see how you specified the colour vertex attributes. The only difference this week is that UV attributes are 2 floats, whereas colour attributes were 3 floats.

Keep in mind that texture coordinates are specified so that Bottom Left of the texture is (0, 0) and Top Right of the texture is (1, 1)



Load the texture into OpenGL

Have a look at line 96 of Lab8.cpp

```
96  | | |   GLuint texture = setup_texture("jubilee.bmp");
```

This is going to be used to set up the texture in OpenGL. Open `texture.h` and you will find the empty function definition

```
GLuint setup_texture(const char* filename)
```

The first thing to do, at the top of the function, is to enable textures

```
|   glEnable(GL_TEXTURE_2D);
|   glEnable(GL_BLEND);
```

Then, generate an OpenGL texture object

```
|   GLuint texObject;
|   glGenTextures(1, &texObject);
|   glBindTexture(GL_TEXTURE_2D, texObject);
```

Next, set some parameters for how OpenGL will draw the texture. Don't worry what these mean for now.

```
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S, GL_REPEAT);
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_T, GL_REPEAT);
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_NEAREST);
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_NEAREST);
```

Then, load the bitmap file, which is at the location stored in the input argument `filename`

```
unsigned char* pxls = NULL;
BITMAPINFOHEADER info;
BITMAPFILEHEADER file;
loadbitmap(filename, pxls, &info, &file);
```

The API function `loadbitmap()` is provided in `bitmap.h`.

```
GLuint loadbitmap(filename, pixelBuffer, infoHeader, fileHeader)
```

`filename` is the file location to of the bitmap to load, `pixelBuffer` is a pointer to the pixel colours which will be updated by the function, `infoHeader` and `fileHeader` are windows structs storing information about bitmap files.

Have a look at `loadbitmap()` if you want to, but the details are not important for this lab.

Next, check that the `pxls` colour data is loaded correctly, and call the OpenGL function `glTexImage2D()`

```
void glTexImage2D(target, level, internalformat, width, height,
border, format, type, data);
```

target should be `GL_TEXTURE_2D`, level will be used later in the lab just set it to 0, internalformat is the number of colour components and we want to use `GL_RGB`, width and height are the dimensions of the image and can be retrieved from the `BITMAPINFOHEADER` struct that was filled in in the `loadbitmap()` function, border must be 0, format should be `GL_RGB`, type should be `GL_UNSIGNED_BYTE`, and data is the colour data we just loaded.

```
if (pxls != NULL)
{
    glTexImage2D(GL_TEXTURE_2D, 0, GL_RGB, info.biWidth, info.biHeight, 0, GL_RGB, GL_UNSIGNED_BYTE, pxls);
}
```

Then, delete the colour data because we don't need it after it has been loaded to OpenGL

```
delete[] pxls;
```

Disable textures

```
glDisable(GL_TEXTURE_2D);
glDisable(GL_BLEND);
```

Finally, return the texture object that OpenGL created

```
    return texObject;
}
```

Back on line 96 of Lab8.cpp see the function `setup_texture()` which you just implemented

```
96  GLuint texture = setup_texture("jubilee.bmp");
```

This will now load the file `jubilee.bmp`, set the pixel data to an OpenGL texture object, and return the texture object. This texture object will need to be bound to OpenGL to be used, which already happens in the render loop.

```
136  glBindTexture(GL_TEXTURE_2D, texture);
137  glUseProgram(shaderProgram);
138  glBindVertexArray(VAO);
139  glDrawArrays(GL_TRIANGLES, 0, 6);
140  glBindVertexArray(0);
```

Set up the VAO and Shaders

Set up the VAO to interpret the vertex attributes correctly

```
unsigned int VAO;
glGenVertexArrays(1, &VAO);
unsigned int VBO;
glGenBuffers(1, &VBO);
glBindVertexArray(VAO);
glBindBuffer(GL_ARRAY_BUFFER, VBO);
glBufferData(GL_ARRAY_BUFFER, sizeof(vertices), vertices, GL_STATIC_DRAW);
glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, 5 * sizeof(float), (void*)0);
glEnableVertexAttribArray(0);
glVertexAttribPointer(1, 2, GL_FLOAT, GL_FALSE, 5 * sizeof(float), (void*)(3 * sizeof(float)));
glEnableVertexAttribArray(1);
glBindBuffer(GL_ARRAY_BUFFER, 0);
glBindVertexArray(0);
//
```

Remember that the position attribute is 3 floats, and the UV attribute is 2 floats.

In the vertex shader, add a new vec3 vertex attribute for the UV, a new vec2 output variable, and copy the UV from the input attribute to the output variable

```
1  #version 330 core
2
3  layout(location = 0) in vec3 aPos;
4  layout(location = 1) in vec3 aTex;
5
6  uniform mat4 model;
7  uniform mat4 view;
8  uniform mat4 projection;
9
10 out vec2 tex;
11
12 void main()
13 {
14     gl_Position = projection * view * model * vec4(aPos, 1.f);
15     tex = aTex.xy;
16 }
```

The line

```
tex = aTex.xy;
```

is something called swizzling which allows the programmer to access components of vectors. In this case, we have copied the x and y components of a vec3 to the vec2 output. Read more about swizzling here [Data Type \(GLSL\) - OpenGL Wiki \(khronos.org\)](https://www.khronos.org/opengl/wiki/Data_Type_(GLSL)) if you're interested.

In the fragment shader, add a new input variable to match the output of the vertex shader, add a texture sampler uniform, and set the colour of the fragment using the texture() sampling function.

```
1  #version 330 core
2
3  out vec4 fragColour;
4
5  in vec2 tex;
6
7  uniform sampler2D Texture;
8
9  void main()
10 {
11     fragColour = texture(Texture, tex);
12 }
13
```

The function texture() on line 11 does the sampling for this fragment.

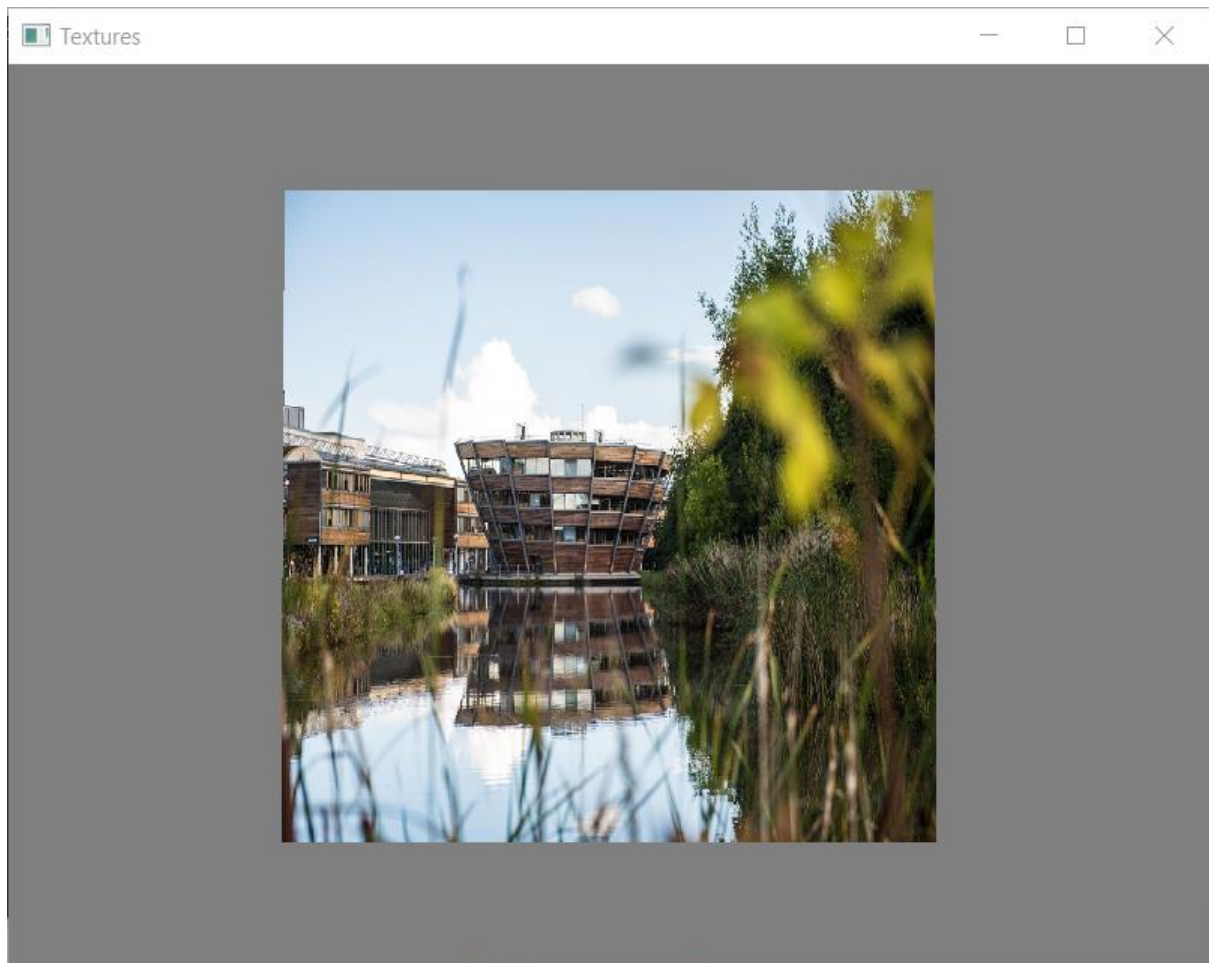
Again, back in Lab8.cpp in the render loop, change the render type from line to filled

```
glClearColor(.5f, .5f, .5f, 1.f);
glClear(GL_COLOR_BUFFER_BIT);
glPolygonMode(GL_FRONT_AND_BACK, GL_FILL);
```

Note that the line to bind the texture is already in the render loop before drawing the triangles.

```
glBindTexture(GL_TEXTURE_2D, texture);
glUseProgram(shaderProgram);
glBindVertexArray(VAO);
glDrawArrays(GL_TRIANGLES, 0, 6);
glBindVertexArray(0);
```

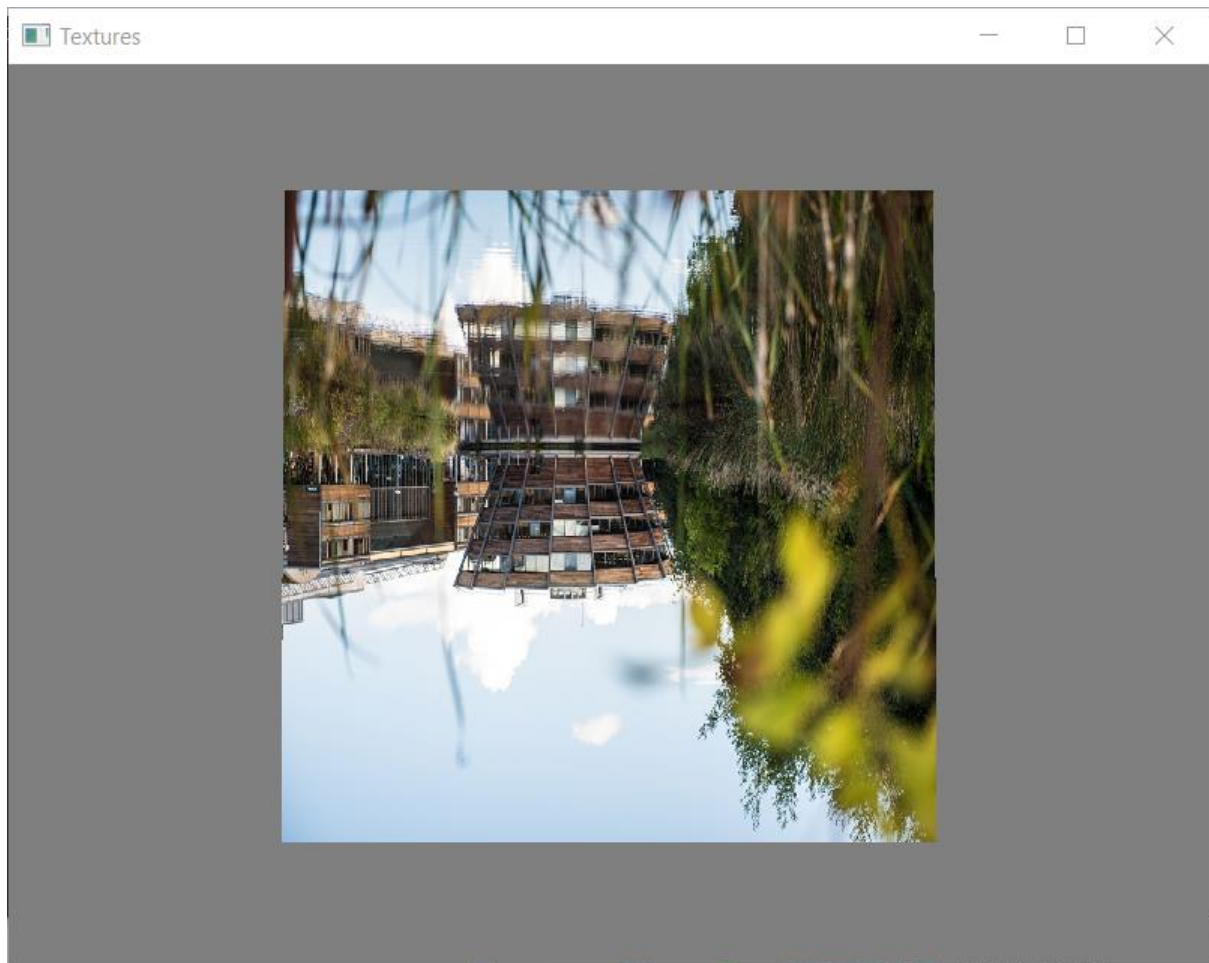
If you run the program you should see the quad has had the image jubilee.bmp texture mapped onto it.



Congratulations! You have now mapped a texture to rendered geometry.

Demo – show one of the teaching team your work.

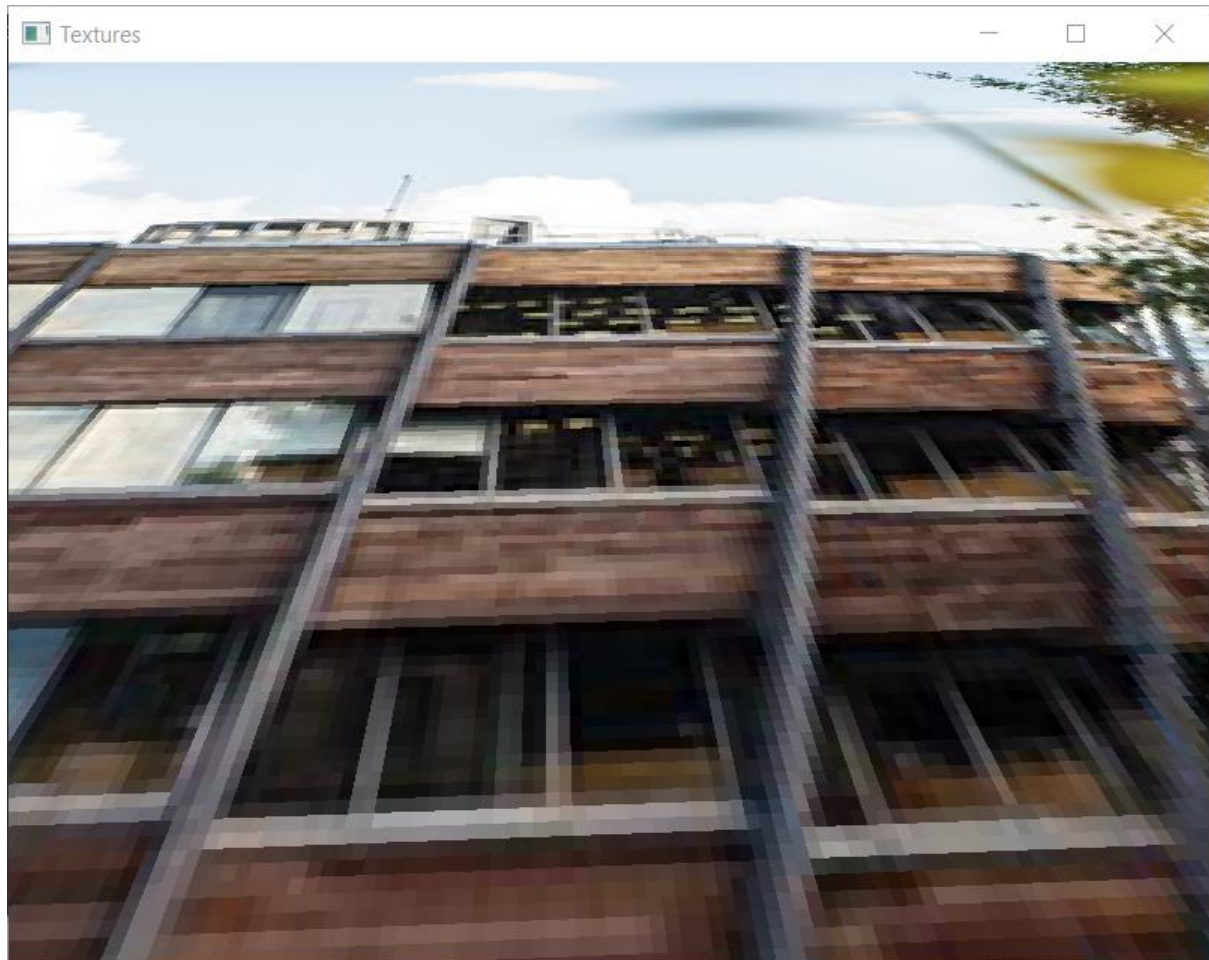
Additional Task – change the UV attributes in the vertex array at the top of Lab8.cpp so that the image is mapped upside down like this



Put the image the right way around again before continuing.

Magnification

Use the LEFT, RIGHT, UP, DOWN, R and F keys to move the ModelViewerCamera around the quad. Put the camera so it is close to the quad and looks something like this



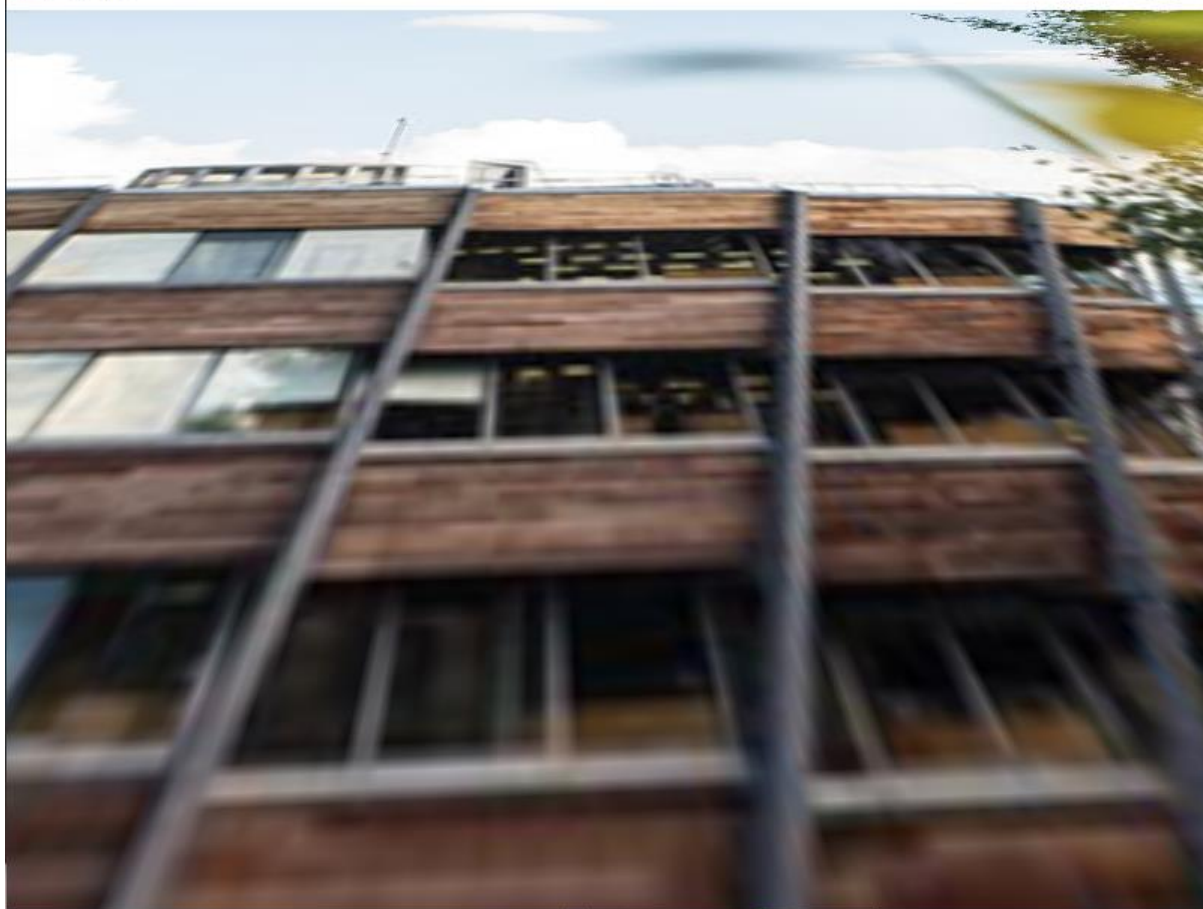
The camera is so close to the quad that the texels are much larger than fragments and magnification is happening. The image has become pixelated.

Enabling Bi-linear filtering is easy. In texture.h set the magnification filter so that it uses `GL_LINEAR`

```
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S, GL_REPEAT);  
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_T, GL_REPEAT);  
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_LINEAR);  
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_NEAREST);
```

Then, when you run the program, and position the camera back in the same spot, you will see a smoother image

Textures



Minification & Mipmaps

To use mipmaps for solving minification, we will need to implement `setup_mipmaps()` in `texture.h`

```
GLuint setup_mipmaps(const char* filename[], int n)
```

This function is very similar to `setup_texture()` which you already have implemented. Just copy/paste the code from the function body of `setup_texture()` into `setup_mipmaps()`.

You will notice that the argument for `setup_texture()` is just a filename, whereas the argument for `setup_mipmaps()` is an array of filenames and a number `n`.

`setup_mipmaps()` is going to do exactly the same as `setup_texture()` but for each of the `n` filenames.

The only change you need to make is to have an array 16 pixel buffers, info and file headers, which will be passed into the `loadbitmap()` function along with the filename parameter.

Then, you need to loop once for each of the `n` filenames and call `loadbitmap()`, check the pixel data is read ok, then call `glTexImage2D` with the specific width, height, and pixel data. One thing to note is that the second parameter of `glTexImage2D()`, `level` is going to be changed in order to change the mipmap level. Finally, delete the individual loaded pixel data.

```
unsigned char* pxls[16];
BITMAPINFOHEADER info[16];
BITMAPFILEHEADER file[16];
for (int c = 0; c < n; c++)
{
    loadbitmap(filename[c], pxls[c], &info[c], &file[c]);

    if (pxls[c] != NULL)
    {
        glTexImage2D(GL_TEXTURE_2D, c, GL_RGB, info[c].biWidth, info[c].biHeight, 0, GL_RGB, GL_UNSIGNED_BYTE, pxls[c]);
    }

    delete[] pxls[c];
}
```

We also need to set the minification filter to trilinear filtering at the top of the `setup_mipmaps()` function.

```
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S, GL_REPEAT);
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_T, GL_REPEAT);
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_LINEAR);
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_LINEAR_MIPMAP_LINEAR);
```

Unzip the

`mipmaps-20220913`

folder and put the `mipmaps` folder into your project directory with the `jubilee.bmp` file.

Inside `mipmaps` folder, you will see 11 files. Open the files and look at them, they are the same image at 11 different sizes.

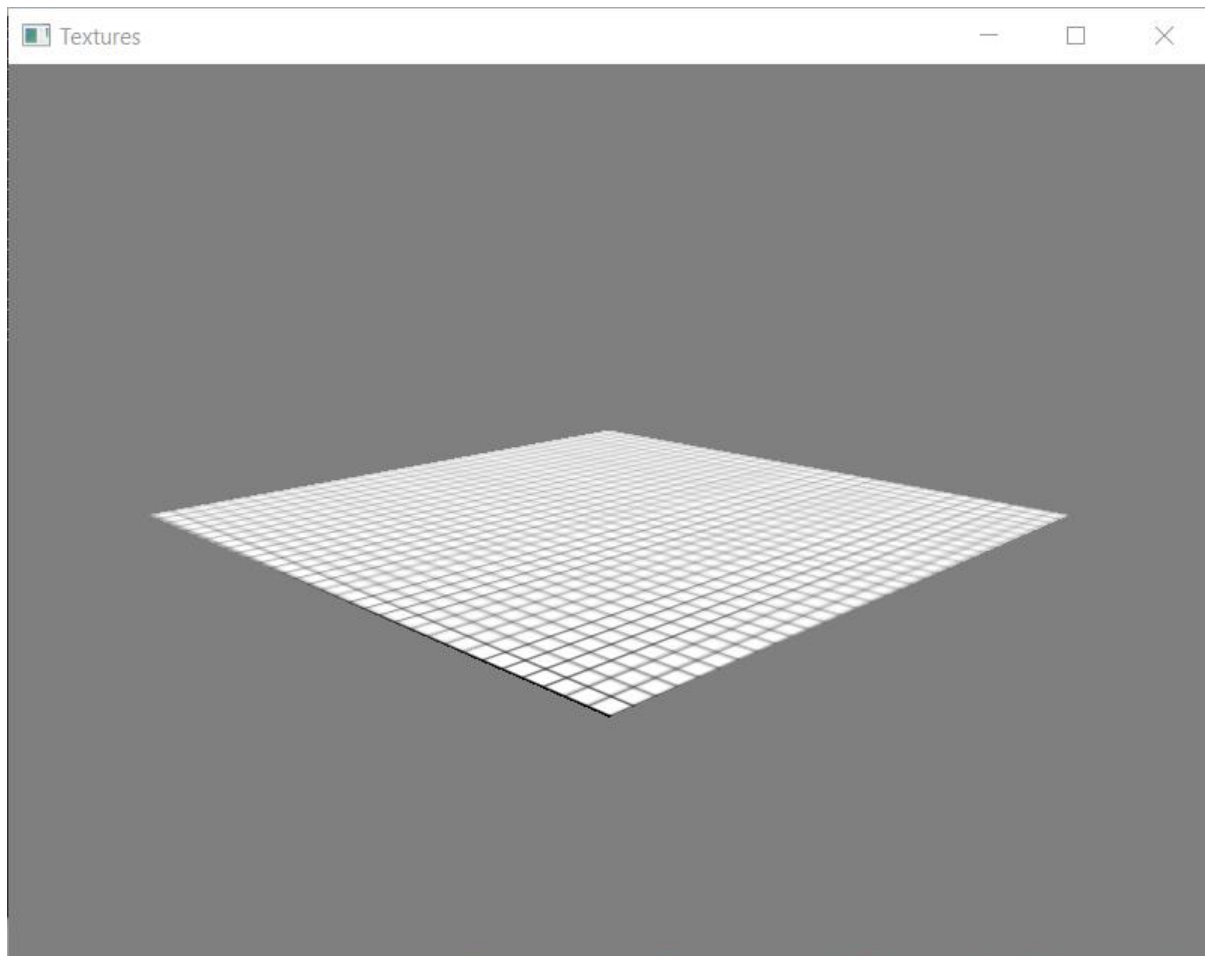
In Lab8.cpp, set up an array of paths to these 11 files and call the new setup_mipmaps() function

```
96     const char* files[11] = {
97         "mipmaps/grid1024x1024.bmp",
98         "mipmaps/grid512x512.bmp",
99         "mipmaps/grid256x256.bmp",
00         "mipmaps/grid128x128.bmp",
01         "mipmaps/grid64x64.bmp",
02         "mipmaps/grid32x32.bmp",
03         "mipmaps/grid16x16.bmp",
04         "mipmaps/grid8x8.bmp",
05         "mipmaps/grid4x4.bmp",
06         "mipmaps/grid2x2.bmp",
07         "mipmaps/grid1x1.bmp"
08     };
09     GLuint texture = setup_mipmaps(files, 11);
10     // GLuint texture = setup_texture("jubilee.bmp");
11
```

Initialise the camera to

```
93     InitCamera(Camera, -45, 15);
94     MoveAndOrientCamera(Camera, glm::vec3(0,0,0), cam_dist, 0.f, 0.f);
95
```

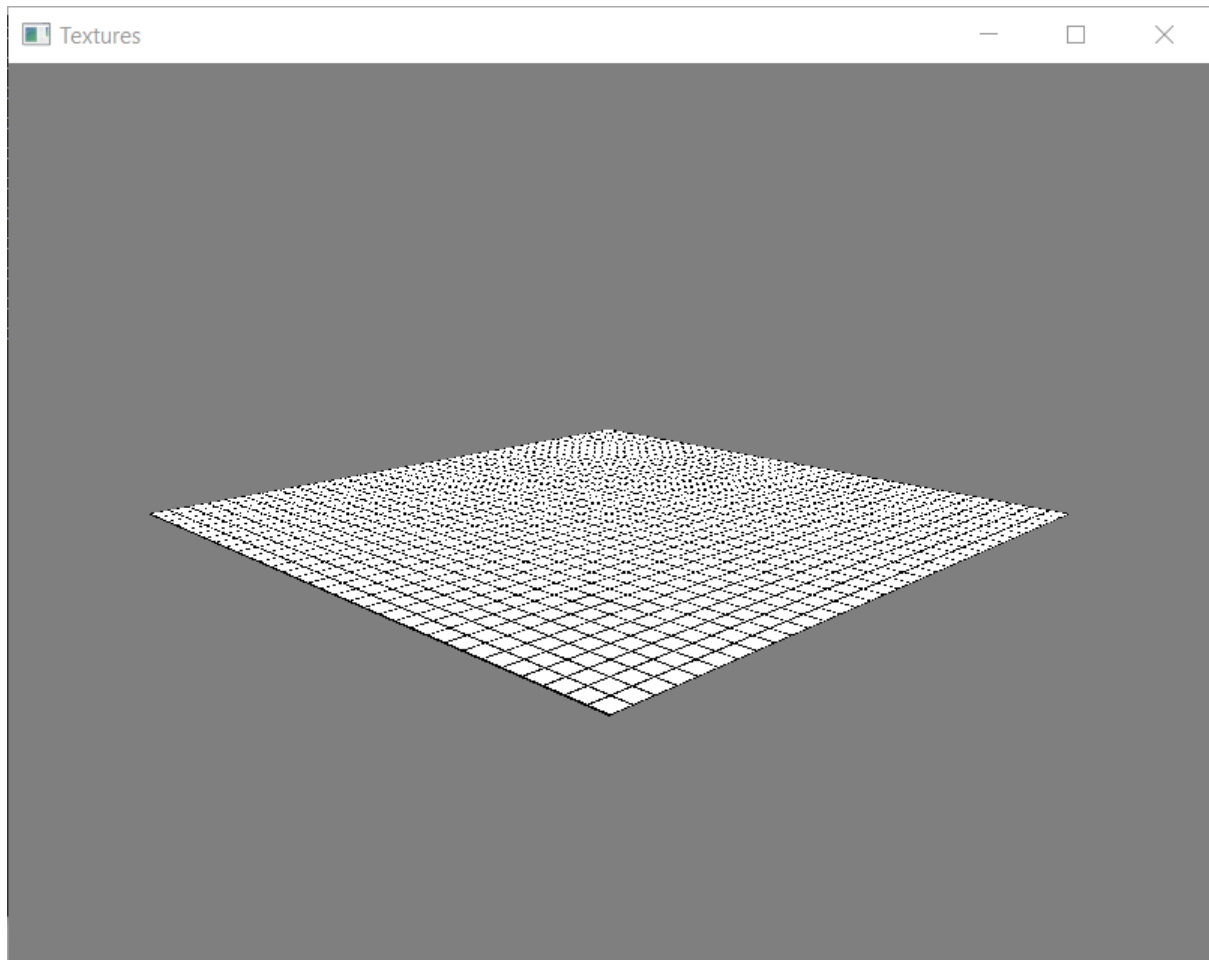
And when you run the program, you will see



In texture.h in the function setup_mipmaps(), turn off trilinear filtering

```
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S, GL_REPEAT);  
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_T, GL_REPEAT);  
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_LINEAR);  
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_NEAREST);
```

And run the program again



You can now see the texture aliasing which was smoothed using the mipmaps.

Toggle trilinear filtering on and off and compare the difference.

And Another Thing...

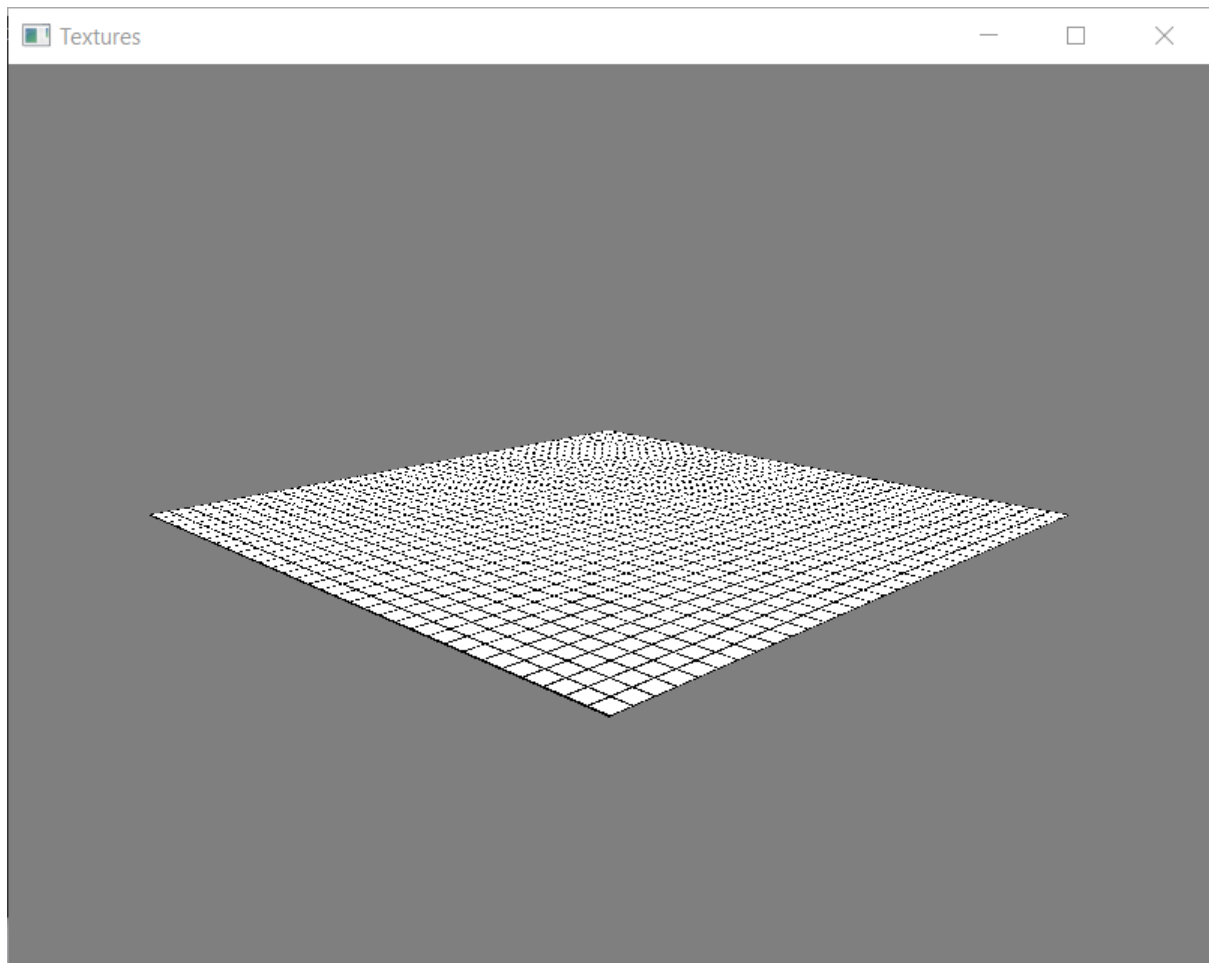
It should be noted that OpenGL has a function for generating mipmaps.

Copy `grid.bmp` to your project folder with `jubilee.bmp`.

In `Lab8.cpp`, comment the code which calls your new `setup_mipmaps()` function, and, instead, call the `setup_texture()` function with the new `grid.bmp` filename, like this

```
1 /*  const char* files[11] = {  
    "mipmaps/grid1024x1024.bmp",  
    "mipmaps/grid512x512.bmp",  
    "mipmaps/grid256x256.bmp",  
    "mipmaps/grid128x128.bmp",  
    "mipmaps/grid64x64.bmp",  
    "mipmaps/grid32x32.bmp",  
    "mipmaps/grid16x16.bmp",  
    "mipmaps/grid8x8.bmp",  
    "mipmaps/grid4x4.bmp",  
    "mipmaps/grid2x2.bmp",  
    "mipmaps/grid1x1.bmp"  
};  
GLuint texture = setup_mipmaps(files, 11);*/  
GLuint texture = setup_texture("grid.bmp");
```

When you run the program you will see this



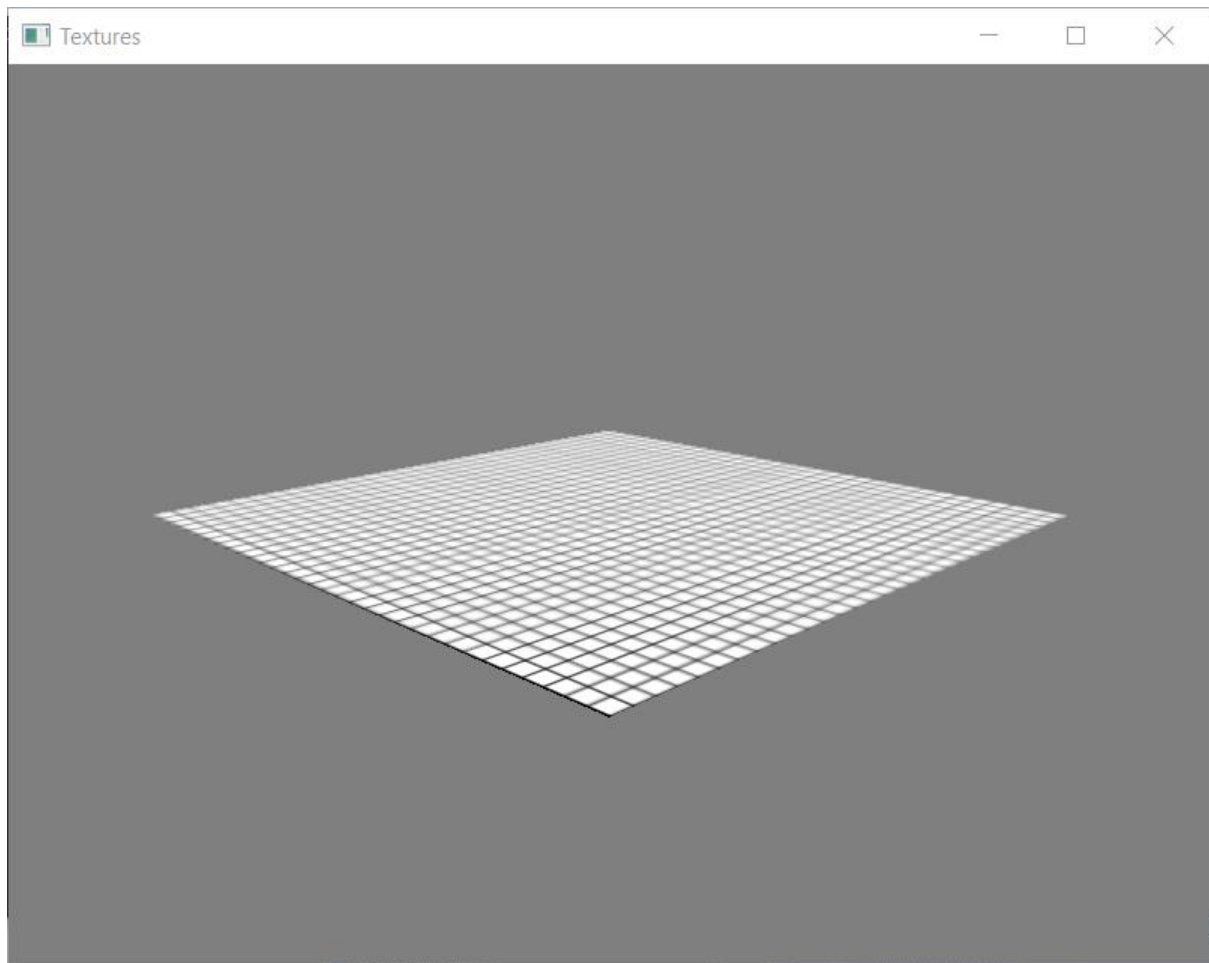
Now, in texture.h, in the function setup_texture(), set minification to trilinear filtering

```
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S, GL_REPEAT);  
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_T, GL_REPEAT);  
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_LINEAR);  
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_LINEAR_MIPMAP_LINEAR);
```

and after calling glTexImage2D(), add call the OpenGL function to generate mipmaps

```
if (pxls != NULL)  
{  
    glTexImage2D(GL_TEXTURE_2D, 0, GL_RGB, info.biWidth, info.biHeight, 0, GL_RGB, GL_UNSIGNED_BYTE, pxls);  
}  
  
glGenerateMipmap(GL_TEXTURE_2D);
```

When you run the program you will see this



We have used an OpenGL function to generate Mipmaps which remove aliasing caused by minification.

The question then is whether to manually create mipmaps, save them to disk and load them using the `setup_mipmaps()` function, or to use the OpenGL function `glGenerateMipmap()` to generate Mipmaps at run time.

The answer is whether you want to save time or space.